CUSTOMER SUPPORT TICKET MANAGEMENT SYSTEM

Project Report

CO 2 AT-3

Name : Shaik Abdul Avez

Reg no:192472260

Course : CSA6503

Technology: Python / Function Calling / MySQL

1. Introduction

Customer service is an important part of every organization that provides products or services. Customers may face problems related to products, payments, accounts, delivery, technical issues, or general enquiries. If these requests are handled through phone calls, emails, or informal messages, it can become difficult to track every complaint and ensure that it is resolved on time.

The Customer Support Ticket Management System provides a centralized platform for recording and managing customer support requests. Each request is converted into a ticket with a unique ticket ID. Support staff can view the ticket, identify its priority, assign it to the appropriate staff member, update its status, and record the final resolution.

In this version, function calling is used to connect user requests with specific system operations. Functions such as create_ticket(), get_ticket_status(), update_ticket(), assign_ticket(), and close_ticket() represent the main actions of the support system.

2. Problem Statement

In a manual customer support process, complaints may be recorded in notebooks, spreadsheets, emails, or separate applications. This can result in duplicate requests, lost complaints, delayed responses, difficulty in finding previous conversations, and poor monitoring of unresolved issues.

The main problem is the absence of a single system that can maintain complete ticket information from the time a customer submits a request until the issue is resolved. A computerized ticket management system using function calling can organize requests and execute the required support operation based on the customer's request.

3. Objectives

- Provide an easy interface for customers to submit support requests.
- Generate a unique ticket ID for every support request.
- Store customer and ticket information securely in a database.
- Use function calling to execute ticket operations based on user requests.
- Allow support staff to view, assign, update, and resolve tickets.
- Track ticket status such as Open, In Progress, Resolved, and Closed.
- Provide search and reporting facilities for support management.

4. Existing System

In a traditional support environment, customers may contact an organization through telephone, email, or direct communication. Staff members manually record the details and forward them to the appropriate department. The customer may need to contact the organization again to know the current status.

The major limitations are manual data entry, poor tracking of pending tickets, difficulty in identifying high-priority issues, limited reporting, and a higher possibility of human error. A centralized function-based system provides a more structured method for performing each ticket operation.

5. Proposed System

The proposed system provides a centralized solution for managing customer issues. Customers can create tickets by entering the subject, description, category, and priority. The system generates a unique ticket number and stores the information in the database.

Function calling allows the application to map a request to a predefined function. For example, when a customer asks for the status of ticket 1005, the application can call get_ticket_status(ticket_id=1005). When a customer wants to create a complaint, create_ticket() can be called with the required details.

This approach separates natural-language requests from actual database operations. Only predefined functions are allowed to perform important actions, making the system easier to maintain and control.

6. System Requirements

6.1 Hardware Requirements

- Processor: Intel Core i3 or higher
- RAM: 4 GB or higher
- Storage: At least 10 GB free space
- Keyboard and mouse
- Display monitor

6.2 Software Requirements

- Operating System: Windows 10/11 or compatible OS
- Python 3.x
- Function-calling capable AI/API environment
- MySQL Server

- MySQL Connector/Python
- IDE such as VS Code or PyCharm

7. System Modules

7.1 Customer Module

The customer module allows users to create tickets, view submitted tickets, check ticket status, and receive responses. Customer information is stored in the database and linked with the tickets created by that customer.

7.2 Ticket Management Module

This module handles ticket creation, retrieval, assignment, updating, and closure. Each ticket contains a ticket ID, customer ID, subject, category, description, priority, date, assigned staff member, and status.

7.3 Function Calling Module

The function calling module is the main intelligent component. It defines a collection of safe, predefined functions. The application identifies which function is required and passes the appropriate arguments. Example functions include create_ticket(), get_ticket(), get_ticket_status(), update_ticket(), assign_ticket(), and close_ticket().

7.4 Staff Module

Support staff can view assigned tickets, examine customer descriptions, communicate responses, change ticket status, and record resolution details. Staff members can search tickets using ticket ID, category, status, or priority.

7.5 Admin Module

The administrator can manage users and staff members and monitor all tickets. The administrator can also review reports and identify unresolved or high-priority issues.

8. Function Calling Design

Function calling provides a structured way for an AI application to request an operation from the backend. Instead of allowing the AI to directly change the database, the application exposes only selected functions. The AI identifies the required function and provides arguments, while the program executes the function and returns the result.

For example, a customer may type: 'Create a ticket because my payment failed.' The system can identify create_ticket() as the required operation and provide arguments such as customer_id, subject, category, priority, and description. The backend function then inserts the ticket into MySQL.

Similarly, the request 'What is the status of ticket 1024?' can be mapped to get_ticket_status(ticket_id=1024). The function retrieves the record and returns the current status. This makes the interaction simple for the user while keeping database operations controlled.

9. Main Functions

- create_ticket(customer_id, subject, category, priority, description) – creates a new support ticket.
- get_ticket(ticket_id) – retrieves complete information about a ticket.
- get_ticket_status(ticket_id) – returns the current status of a ticket.
- update_ticket(ticket_id, status, response) – updates ticket status and response.
- assign_ticket(ticket_id, staff_id) – assigns a ticket to a support staff member.
- close_ticket(ticket_id, resolution) – records the resolution and closes the ticket.

10. Function Calling Workflow

1. Customer enters a request in natural language.
2. The application sends the request to the function-calling model.
3. The model identifies the required predefined function.
4. The model supplies the function arguments.
5. The application validates the arguments.
6. The selected function performs the database operation.
7. The function returns the result to the application.
8. The result is displayed to the customer.

11. Database Design

MySQL can be used to store all application data. A relational database connects customers, staff members, tickets, and responses while maintaining consistent information.

Table	Important Fields	Purpose	Key
Customer	customer_id, name, email, password	Stores customer accounts	customer_id
Staff	staff_id, name, email, password	Stores support staff	staff_id
Ticket	ticket_id, customer_id, subject, category, priority, status	Stores support requests	ticket_id
Response	response_id, ticket_id, staff_id, message, date	Stores ticket communication	response_id
Category	category_id, category_name	Stores issue categories	category_id

12. Example Function Calling Logic

A simplified implementation can define Python functions for database operations. The function-calling layer selects one of these functions based on the user's request. For example, the backend can contain create_ticket() and get_ticket_status() functions, while the AI layer decides which function should be called.

The important design principle is that the model does not directly execute arbitrary SQL commands. It requests one of the allowed functions, and the application validates the parameters before performing the operation. This improves reliability and makes the application easier to test.

13. Advantages

- Centralized management of customer support requests.
- Natural-language interaction with the support system.
- Controlled execution through predefined functions.
- Faster identification of pending and high-priority issues.

- Reduced manual record keeping.
- Easy tracking of ticket status and resolution history.
- Search and reporting make support management easier.
- The design can be extended with additional functions.

14. Applications

The system can be used by software companies, educational institutions, online shopping businesses, banks, telecom service providers, hospitals, hotels, and other organizations that receive customer complaints or service requests. Function calling is particularly useful when users want to interact with the system using natural language.

15. Future Enhancements

The system can be enhanced with email and SMS notifications, file attachments, automatic ticket assignment, service-level agreement alerts, customer satisfaction ratings, and graphical dashboards.

AI-based ticket classification can automatically identify categories and priorities. Additional functions can be added for refund requests, escalation, knowledge-base search, appointment scheduling, and FAQ responses. A web or mobile interface can also provide remote access.

16. Conclusion

The Customer Support Ticket Management System provides an organized method for handling customer complaints and service requests. By converting every request into a trackable ticket, the system makes it easier to assign work, monitor progress, communicate with customers, and record resolutions.

The use of function calling makes the system more interactive and structured. Natural-language requests can be converted into calls to predefined backend functions such as create_ticket(), get_ticket_status(), update_ticket(), and close_ticket(). With Python and MySQL, the project demonstrates practical concepts in application development, database management, and AI-assisted automation.

17. References

- Python Documentation – Python programming and database connectivity concepts.
- MySQL Documentation – SQL and relational database concepts.
- OpenAI API documentation – Function/tool calling concepts.